Real-Time Crypto Prices with Kafka on AWS

Hands-on lab guide: Binance API to Kafka (EC2) to PostgreSQL (RDS)

1. What you will build

A streaming pipeline that collects live cryptocurrency prices and stores them in a database you can query with SQL.

```
Binance API -> Python producer -> Kafka topic (3 partitions) -> Python
consumers -> RDS PostgreSQL -> SQL client
```

- The producer polls Binance every 2 seconds for BTC, ETH, SOL and BNB prices.
- Kafka runs on one EC2 instance. The topic crypto_prices has 3 partitions and each message is keyed by coin symbol.
- Consumers in one consumer group read the topic in parallel and insert rows into PostgreSQL in batches of 50.
- You query the table with SQL from DBeaver or pgAdmin.

By the end you will be able to:

- Start Zookeeper and Kafka on EC2 and expose the broker safely to your laptop.
- Create a topic with several partitions and explain how message keys choose a partition.
- Run several consumers in one group and watch partitions being shared and rebalanced.
- Load streaming data into PostgreSQL and analyse it with SQL.

Estimated time: about 2 hours for the first run.

2. Before you start

Checklist:

- An AWS account, working in the Europe (Stockholm) region.
- An EC2 instance with Java installed and Kafka 3.3.1 extracted in the folder kafka_2.12-3.3.1.
- Python 3.11 on your laptop, with Jupyter or VS Code.

- DBeaver or pgAdmin installed on your laptop.
- Your laptop's public IP address (search the web for 'what is my IP').

Install the Python packages on your laptop:

```
pip install kafka-python requests psycopg2-binary
```

Cost and safety rules for this lab:

- Create a budget alert first: Billing and Cost Management, Budgets, Create budget.
- Do not leave an 'All traffic from 0.0.0.0/0' rule on your security group. Open only the ports in this guide, only to your own IP.
- Delete or stop everything when you finish (Section 11).

Placeholders used in this guide:

- EC2_PUBLIC_IP: the public IPv4 address of your EC2 instance. It changes if you stop and start the instance, unless you attach an Elastic IP.
- RDS_ENDPOINT: the database endpoint you will copy in Section 6.

3. Part 1: Start Kafka on EC2

You will need three SSH terminals to the EC2 instance. Call them Terminal A (Zookeeper), Terminal B (Kafka) and Terminal C (admin commands).

Step 1.1 Connect to EC2

```
ssh -i your-key.pem ec2-user@EC2_PUBLIC_IP
```

Repeat in two more windows so you have Terminals A, B and C.

Step 1.2 Start Zookeeper (Terminal A)

```
cd kafka_2.12-3.3.1
bin/zookeeper-server-start.sh config/zookeeper.properties
```

Leave it running.

Step 1.3 Tell Kafka its public address (Terminal C)

Kafka replies to clients with the address in advertised.listeners. If this is wrong, your laptop connects once and then times out.

```
cd kafka_2.12-3.3.1
nano config/server.properties
```

Find the advertised.listeners line, remove any leading # and set it to your public IP:

```
advertised.listeners=PLAINTEXT://EC2_PUBLIC_IP:9092
```

Save with Ctrl+O, Enter, then exit with Ctrl+X.

Step 1.4 Start Kafka (Terminal B)

```
cd kafka_2.12-3.3.1
export KAFKA_HEAP_OPTS='-Xmx256M -Xms128M'
bin/kafka-server-start.sh config/server.properties
```

The heap setting keeps Kafka within the memory of a small instance. Leave it running. If you edit server.properties later, stop Kafka with Ctrl+C and start it again, because the file is only read at startup.

Step 1.5 Open port 9092 (AWS console)

1. Go to EC2, Instances, select your instance, then the Security tab, then click the security group.
2. Edit inbound rules and add: Type Custom TCP, Port 9092, Source My IP.
3. Save the rules.

Checkpoint 1

On EC2 (Terminal C):

```
ss -tlnp | grep 9092
```

You should see Kafka listening on port 9092. From your laptop, run this in Python:

```
import socket
s = socket.socket()
s.settimeout(5)
print('open' if s.connect_ex(('EC2_PUBLIC_IP', 9092)) == 0 else 'blocked')
s.close()
```

It must print open before you continue.

4. Part 2: Create the topic with 3 partitions

On EC2 (Terminal C):

```
cd kafka_2.12-3.3.1
bin/kafka-topics.sh --create --topic crypto_prices --bootstrap-server
EC2_PUBLIC_IP:9092 --replication-factor 1 --partitions 3
bin/kafka-topics.sh --describe --topic crypto_prices --bootstrap-server
EC2_PUBLIC_IP:9092
```

The describe output lists Partition 0, 1 and 2. Replication factor is 1 because we have only one broker. Partitions can be increased later but never decreased.

5. Part 3: Write and run the producer

On your laptop, create producer.py (or a notebook cell). Replace EC2_PUBLIC_IP.

```
import json, time, requests
from kafka import KafkaProducer

BROKER = 'EC2_PUBLIC_IP:9092'
TOPIC = 'crypto_prices'
SYMBOLS = ['BTCUSDT', 'ETHUSDT', 'SOLUSDT', 'BNBUSDT']
URL = 'https://api.binance.com/api/v3/ticker/price'

producer = KafkaProducer(
    bootstrap_servers=[BROKER],
    key_serializer=lambda k: k.encode('utf-8'),
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
)

while True:
    resp = requests.get(URL, params={'symbols': json.dumps(SYMBOLS,
```

```
separators=(',', ':'))}, timeout=5)
    resp.raise_for_status()
    now = time.strftime('%Y-%m-%d %H:%M:%S', time.gmtime())
    for item in resp.json():
        event = {'symbol': item['symbol'], 'price': float(item['price']),
'ts': now}
        meta = producer.send(TOPIC, key=item['symbol'],
value=event).get(timeout=10)
        print(item['symbol'], item['price'], '-> partition', meta.partition,
'offset', meta.offset)
    time.sleep(2)
```

Run it. You should see a new line for each coin every 2 seconds. Notice that each coin always goes to the same partition. That is the key at work. With 4 coins and 3 partitions, two coins may share a partition.

Stop it with Ctrl+C (or the stop button in Jupyter) when you want to pause.

6. Part 4: Verify messages in Kafka

With the producer running, on EC2 (Terminal C):

```
bin/kafka-console-consumer.sh --topic crypto_prices --bootstrap-server
EC2_PUBLIC_IP:9092 --from-beginning
```

You should see JSON messages scrolling. Stop it with Ctrl+C. Do not continue until you see data here.

7. Part 5: Create the PostgreSQL database (RDS)

Step 5.1 Create the instance

1. In the AWS console search for RDS and open Aurora and RDS, then click Create database.
2. Choose Standard create (or Easy create if you prefer, then edit the settings below).
3. Engine type: PostgreSQL. Pick plain PostgreSQL, not Aurora, because Aurora Serverless pauses itself and the first connection after a pause can time out.
4. Template: Free tier (or Sandbox, whichever your account offers).
5. Availability: Single-AZ DB instance.

6. DB instance identifier: crypto-db. Master username: postgres. Choose a password and write it down.
7. Instance class: db.t3.micro (or db.t4g.micro).
8. Storage: General purpose SSD, 20 GiB. Untick storage autoscaling.
9. Connectivity: Public access Yes. VPC security group: create new, named rds-postgres-sg.
10. Additional configuration: Initial database name crypto. Turn off Performance Insights and Enhanced Monitoring. Set backup retention to 1 day.
11. Check the Estimated monthly costs panel, then click Create database. It takes 5 to 10 minutes.

Step 5.2 Open port 5432

EC2, Security Groups, rds-postgres-sg, Edit inbound rules: Type PostgreSQL, Port 5432, Source My IP.

Step 5.3 Copy the endpoint

RDS, Databases, crypto-db, Connectivity and security. Copy the Endpoint. It looks like crypto-db.xxxx.eu-north-1.rds.amazonaws.com. This is your RDS_ENDPOINT.

Step 5.4 Create the table

Connect with DBeaver or pgAdmin (host RDS_ENDPOINT, port 5432, database crypto, user postgres, SSL enabled) and run:

```
CREATE TABLE IF NOT EXISTS crypto_prices (
  symbol          VARCHAR(20),
  price           DECIMAL(18,8),
  event_ts        TIMESTAMP,
  kafka_partition INT,
  kafka_offset    BIGINT
);
```

Checkpoint 2

Run SELECT COUNT(*) FROM crypto_prices; It should return 0 with no errors.

8. Part 6: Write and run the consumer

Create consumer.py on your laptop. Replace the three placeholders.

```
import json, time
import psycopg2
from psycopg2.extras import execute_values
from kafka import KafkaConsumer

BROKER = 'EC2_PUBLIC_IP:9092'
DB = dict(host='RDS_ENDPOINT', port=5432, dbname='crypto',
          user='postgres', password='YOUR_PASSWORD',
          sslmode='require', connect_timeout=15)

def connect_db(retries=6, wait=10):
    for attempt in range(1, retries + 1):
        try:
            return psycopg2.connect(**DB)
        except psycopg2.OperationalError as e:
            print('DB not ready', attempt, '/', retries, ':', e)
            time.sleep(wait)
    raise RuntimeError('Could not connect to the database')

conn = connect_db()
cur = conn.cursor()

consumer = KafkaConsumer(
    'crypto_prices',
    bootstrap_servers=[BROKER],
    group_id='crypto-loader',
    auto_offset_reset='earliest',
    enable_auto_commit=False,
    value_deserializer=lambda x: json.loads(x.decode('utf-8')),
)

BATCH_SIZE = 50
batch = []
for msg in consumer:
    v = msg.value
    batch.append((v['symbol'], v['price'], v['ts'], msg.partition,
msg.offset))
    if len(batch) >= BATCH_SIZE:
        execute_values(cur,
```

```
            'INSERT INTO crypto_prices (symbol, price, event_ts,
kafka_partition, kafka_offset) VALUES %s',
            batch)
        conn.commit()
        consumer.commit()
        print('loaded', len(batch), 'rows; my partitions:',
              sorted(p.partition for p in consumer.assignment()))
        batch.clear()
```

Why it is written this way:

- Rows are inserted in batches of 50, not one at a time, which is far faster.
- Kafka offsets are committed only after the database commit succeeds. If the consumer crashes, Kafka redelivers the uncommitted messages, so no data is lost (a few rows may be duplicated).
- group_id puts every copy of this script in the same consumer group, so they share the partitions.
- The retry loop handles a database that is slow to respond.

Start the producer, then the consumer. With 4 messages every 2 seconds, the first batch loads after roughly 25 seconds.

Checkpoint 3

In your SQL client run SELECT COUNT(*) FROM crypto_prices; The number should grow in steps of 50.

9. Part 7: Parallel processing demo

This is the key Kafka lesson: partitions are the unit of parallelism, and consumers in one group divide them.

1. Stop the consumer. Start one copy (Terminal 1). Its printed partitions should be [0, 1, 2].
2. Start a second copy (Terminal 2) with the same group_id. After a few seconds both copies rebalance. Their partitions are now split, for example [0, 1] and [2].
3. Start a third copy. Each copy owns one partition: [0], [1], [2].
4. Start a fourth copy. It receives no partitions and sits idle, because a partition can be read by only one consumer in a group.
5. Stop one running copy with Ctrl+C. After about 10 to 30 seconds the others take over its partition (rebalancing).

To see how work was spread, run this query:

```
SELECT kafka_partition, COUNT(*) AS rows_loaded
FROM crypto_prices
GROUP BY kafka_partition
ORDER BY kafka_partition;
```

Discussion questions:

- Why is the number of useful consumers limited by the number of partitions?
- Why do all messages for one coin stay in order?
- What would change if you used a random key, or no key?

10. Part 8: Analyse the data with SQL

```
-- Latest price per coin
SELECT symbol, price, event_ts FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY symbol ORDER BY event_ts DESC) AS
rn
  FROM crypto_prices) t
WHERE rn = 1;

-- One-minute statistics
SELECT symbol, DATE_TRUNC('minute', event_ts) AS minute,
       MIN(price) AS low, MAX(price) AS high, AVG(price) AS avg_price,
COUNT(*) AS ticks
FROM crypto_prices
GROUP BY 1, 2
ORDER BY 2 DESC, 1;

-- Biggest price swing per coin
SELECT symbol, MAX(price) - MIN(price) AS swing
FROM crypto_prices
GROUP BY symbol
ORDER BY swing DESC;

-- Remove old rows (the free-tier database has limited storage)
DELETE FROM crypto_prices WHERE event_ts < NOW() - INTERVAL '1 day';
```

11. Troubleshooting

- KafkaTimeoutError: Failed to update metadata after 60 secs. Check advertised.listeners uses the current public IP, restart Kafka after editing it, and confirm port 9092 is open to your IP.
- NoBrokersAvailable. Kafka is not running, the IP is wrong (it changes when the instance restarts), or port 9092 is blocked.
- Binance returns HTTP 451 or 403. Binance restricts some regions. Try https://data-api.binance.vision/api/v3/ticker/price with the same parameters, or use CoinGecko (https://api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum&vs_currencies=usd) and poll every 15 to 30 seconds.
- psycopg2 connection timeout. Check Public access is Yes, port 5432 is open to your current IP (home IPs change), and the database status is Available.
- password authentication failed. Re-check the password, or reset it with Modify on the RDS instance.
- database crypto does not exist. You skipped the initial database name. Connect to the postgres database and run CREATE DATABASE crypto;
- SSL errors. Keep sslmode set to require.
- No rows appear in the table. The batch of 50 may not be full yet; wait about 30 seconds. Confirm the producer is running and the console consumer shows messages.
- An import error mentioning six.moves with kafka-python. Upgrade with pip install -U kafka-python.

12. Clean up (do not skip)

1. Stop the producer and all consumers.
2. RDS: select crypto-db, Actions, Delete. Untick the final snapshot and automated backups, confirm.
3. EC2: stop the instance (or terminate it if the lab is finished). Stopping changes its public IP unless you use an Elastic IP.
4. Security groups: delete the rules you added if you no longer need them. Remove any rule that allows all traffic from 0.0.0.0/0.
5. Billing: confirm in a day or two that no unexpected charges appear.

13. Extensions and assignments

- Add two more coins and observe how keys spread across partitions.
- Increase the topic to 6 partitions and run 6 consumers. What changes?
- Add a second consumer group (for example price-alerts) that prints a warning when a coin moves more than 0.5 percent in a minute, and show that both groups read the same stream.
- Write batches to S3 as Parquet instead of PostgreSQL.
- Move the broker, producer and consumers into private subnets in a VPC, and keep the database private.
- Replace Zookeeper with KRaft mode on a newer Kafka version.